

IRSKG: Unified Intrusion Response System Knowledge Graph Ontology for Cyber Defense

Damodar Panigrahi[†], Shaswata Mitra[‡], Subash Neupane[§], Sudip Mittal[¶], Benjamin A. Blakely[¶],
Dept. of Computer Science & Engineering, Mississippi State University, Starkville, MS, USA
{dp1657, sm3843, sn922}@msstate.edu, mittal@cse.msstate.edu
Argonne National Laboratory, Ankeny, IA, USA
bblakely@anl.gov

Abstract—Cyberattacks are becoming increasingly difficult to detect and prevent due to their sophistication. In response, Autonomous Intelligent Cyber-defense Agents (AICAs) are emerging as crucial solutions. One prominent AICA agent is the Intrusion Response System (IRS), which is critical for mitigating threats after detection. IRS uses several Tactics, Techniques, and Procedures (TTPs) to mitigate attacks and restore the infrastructure to normal operations. Continuous monitoring of the enterprise infrastructure is an essential TTP the IRS uses. However, each system serves different purposes to meet operational needs. Integrating these disparate sources for continuous monitoring increases pre-processing complexity and limits automation, eventually prolonging critical response time for attackers to exploit. We propose a unified IRS Knowledge Graph ontology (IRSKG) that streamlines the onboarding of new enterprise systems as a source for the AICAs. Our ontology can capture system monitoring logs and supplemental data, such as a rules repository containing the administrator-defined policies to dictate the IRS responses. Besides, our ontology permits us to incorporate dynamic changes to adapt to the evolving cyber-threat landscape. This robust yet concise design allows machine learning models to train effectively and recover a compromised system to its desired state autonomously with explainability.

Index Terms—Cybersecurity, Intrusion Response System (IRS), Knowledge Graph, Ontology, Artificial Intelligence (AI)

I. INTRODUCTION

Most of today's automated cyber defense tools are passive watchers and do little to plan and execute responses to attacks, as well as recovery activities [1]. Response and recovery are the two core components of cyber resilience and are left for human cyber analysts, incident responders, and system administrators. Given the escalating threats, Autonomous Intelligent Cyber-defense Agents (AICAs) have emerged as a security mechanism that offers adaptive and real-time protection against evolving digital threats. AICAs leverage AI and Machine Learning (ML) techniques to independently monitor network traffic, identify anomalies, and respond to potential security breaches without continuous human intervention. Specifically, AICA, with its Intrusion Detection System (IDS) and Intrusion Response System (IRS) components, is designed to automatically identify and initiate the most effective response to an ongoing attack [2]. IDS identifies potential security breaches or attacks by monitoring network traffic and system activities. The IRS then dynamically adjusts its defense strategies based on the identified threat's nature, effectively

diluting the impact before it can cause significant damage while restoring the system to its desired state.

To effectively respond to an active attack, the IRS requires data from various sources, such as IDS and enterprise sensors, to identify suitable Rules of Engagement (RoEs) and determine intended behavior. This multi-source data is then used with RoEs to further train the AI and ML models for predictions, enhancing the dynamic and real-time fortifications against these attacks. As a result, the IRS must process system logs, RoEs, and AI/ML model input data for learning and prediction. However, enterprise systems possess individual schemas, leading to complex knowledge propagation and increased inference time. This multi-faceted data ingestion creates several problems (listed below) that significantly impede cyber defense operations.

- Firstly, AICAs need to interact and share information seamlessly. Diverse operating schema between multiple systems often results in data misinterpretation and complicates IRS AI/ML training and predictive modeling. Without proper training, ML systems cannot differentiate between benign and legitimate threats, increasing false positives or unnecessary alerts and responses.
- Secondly, cyber defense is a collaborative effort where organizations, governmental bodies, and security agencies collaborate together to address threats across organizational boundaries. Having different communication schemas to share threat intelligence and response strategies prolongs information sharing.
- Lastly, the cyber threat landscape is dynamic, meaning attack vectors, tools, and defense mechanisms evolve, and the IRS must frequently adapt to new threat patterns. Without a standardized new information ingestion channel, agility is compromised, and various regulatory compliance audits for detecting, responding, and reporting cyber incidents are prevented.

Unfortunately, due to these limitations, Intrusion Response Systems (IRS) have not been able to keep up with the increasingly developed a knowledge graph ontology that can encompass senses and strategies from various sources. By representing knowledge from diverse sources in a unified manner, we can enhance the AICA's efficiency in prompt response and

recovery. The same ontology can also capture the RoEs and security [7], information system risk management [8], cyber AI/ML model input data, as shown in Fig. 1. Therefore, this work introduces a revolutionary AICAs IRS Knowledge Graph (IRSKG) ontology that allows streamlined knowledge ingestion, sharing, and adaptation. We demonstrate our ontology implementation utilizing a case study (see Section IV for details): a network infrastructure management enterprise system. To validate our approach, we demonstrated improved IRS representation techniques for AI/ML models. Due to the graphical structure, we demonstrate a Graph Neural Network (GNN) representation for defensive cyber operations [4] using the IRS. It is important to note that our generalized ontology is designed to accommodate any techniques, AI/ML models of choice, and enterprise systems. As per our knowledge, the first research is the first attempt to develop a unified knowledge graph ontology for IRS systems.

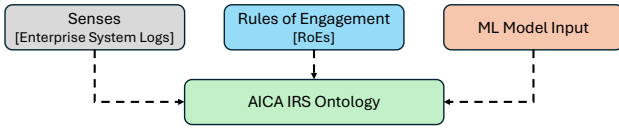


Figure 1: AICA Intrusion Response System Knowledge Graph (IRSKG) ontology to store different senses: enterprise system logs, Rules of Engagements (RoEs), and AI/ML model input.

The rest of the paper is organized as follows - Section II presents background information and related work pertinent to graph notation. In addition, we introduce a prototype in Section II-B that we use to demonstrate our ontology. Next, we define the ontology, AICA-IRS-KG, in Section III. Following the ontology definition, we present the Case Study demonstrating it by choosing the prototype in Section IV. Finally, we conclude the paper in Section V.

II. BACKGROUND

In this section, we cover the preliminaries of our ontology. We cover popular graph semantics to build an ontology for the AICA IRS system. We extensively evaluate two popular graph model techniques, namely *Resource Description Framework (RDF)* and *Property Graph (PG)* in Section II-A. Then, we briefly describe AICA, IRS, IRS rules, IRS computation model, and AICA prototype in Section II-B.

A. Knowledge Representation, Knowledge Graph Ontologies, and Property Graphs (PG)

Knowledge Representation (KR) organizes information in a way the computer software can understand and use to solve specific tasks. KR captures real-world knowledge so that software can process and reason on it. *Ontologies* are specialized approaches of KR and act as blueprints to represent knowledge in specific domains. They define the entities and their relationships in a particular domain relevant only to that domain. *Property Graph (PG)* [5] schema handles complex and evolving relationships defined by the ontologies. We found a few ontologies on security alert systems [6], smart city

here. The specification uses a circle to represent a 'vertex' having a 'label' which identifies it. An arrow represents an edge between two vertices with an identity represented by a 'label'. A rectangle represents multiple key-value pairs for vertices and edges.

We demonstrate the PG specification by showing a TCP packet flow between a Web browser and an intranet-hosted Web server. Figure 2 captures a partial network flow of the TCP packets from the Web browser to the Web server. The Graph model depicts the TCP packet flow starting from a Web Browser that attempts to get home.html hosted in mywebserver.com using HTTP protocol on port 8080. The Computer, where the Web Browser is running, then contacts a domain Name Service, to resolve the mywebserver.com to a destination address. The Computer then forwards the request to a network Router with the destination address, which eventually connects to the Web Server. An example of two vertices connected via an edge: the vertex Web Browser (properties: host = mywebserver.com, port = 8080), protocol = http, page = home.html has an edge accessPage (properties: pid = 34567) to another vertex computer (properties: ip = 1.2.3.4, os = linux).

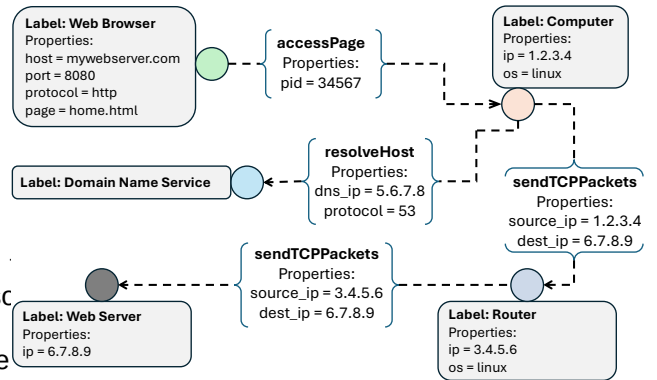


Figure 2: Graph model built using Property Graph (PG), also called Labeled Property Graph (LPG). A partial yet simple illustration of a TCP packet flow between a Web browser and an Intranet-hosted Web Server.

Another specification schema is the Resource Description Framework (RDF) [13]. It draws its inspiration from organizing information on the Web. Its genesis lies in capturing the relationships among different web pages on the Internet. Its eloquence lies in the fact that the model accommodates changes relatively quickly when a new relationship emerges without modifying a substantial portion of the graph. The first specification, RDF 1.0, was published in 1994 [14], and soon after, Open Web Ontology (OWL) and Simple Knowledge Organization System (SKOS) specifications were built on top of RDF. We chose PG as the foundational schema for our Intrusion Response Knowledge Graph ontology over RDF for reasons that we discuss in Section III.

B. Intrusion Response System (IRS)

AICAs defend enterprise systems from cybersecurity breaches [15]. The AICAs are a group of security software that collectively continuously monitors systems and detects and remedies security breaches. One such instance is the Intrusion Response System (IRS). It restores systems to their desired state during a cyberattack, as shown in Figure 3.

The IRS is an AICA component responsible for thwarting security breaches. It automates the procedure of responding to cybersecurity breaches to save time and reduce damage. IRS aims to prevent an attack and restore breached enterprise systems to their desired state by following certain enterprise administrators' defined IRS governing rules. The primary two IRS *Plan* and *Execute* components collectively fulfill IRS objectives. They use rules as a guiding principle to generate actions to thwart security breaches. Moreover, the IRS Plan component uses these rules and the system logs to train an AI/ML model that generates a response to a security breach.

To meet its goal, an IRS computes a *variety of potential responses* including taking actions to prevent the attack from completing, restoring the system to comply with the organizational security policy, containing or confining an attack, attack eradication, deploying forensics measures to enable future attack analysis, counterattack, etc. IRS depends on governing rules defined by administrators to compute responses. There are two primary category rules governing an IRS. The *first category* tells the IRS when to trigger such actions. The IRS Plan and Execute components use these rules. However, such a system must have defined rules to constraining its actions. They are the *second rule category*, called Rules of Engagement (ROEs). Systems must determine which actions they can take in a fully automated manner (and when), which actions require confirmation from a human operator, and which actions must never be executed. The IRS-constrained action components use ROEs. We define the rules, their semantics, and templates, with an illustration in Section III-B.

The IRS Plan component uses different techniques, such as game theory, machine learning, etc., to create computational models. It utilizes the rules and the enterprise system information logs to build the models. The component employs

these pre-trained models to generate the best response to thwart security breaches and restore the enterprise system to its desired state defined by the administrators. We use an *AICA prototype*, as shown in Figure 3, to demonstrate our ontology implementation in the Case Study Section. Next, we describe the prototype data flow. First, the *AICA-Monitoring* managing system components continuously monitor the enterprise systems, thereby collecting the data from AICA sensors and storing data in the *AICA-Knowledge* components. Second, the *AICA-Analyze* components continuously investigate the stored data to detect threats. Third, the *AICA-Plan* component creates a plan for each threat, to thwart cyberattacks. Fourth, the *AICA-Constrained Action* component receives a response from the Plan component, checks if the response is permitted as per the ROEs, creates a final response and passes it forward. Fifth, the *AICA-Execute* component executes the response on the enterprise system(s) through AICA actuators to restore the system to its desired state.

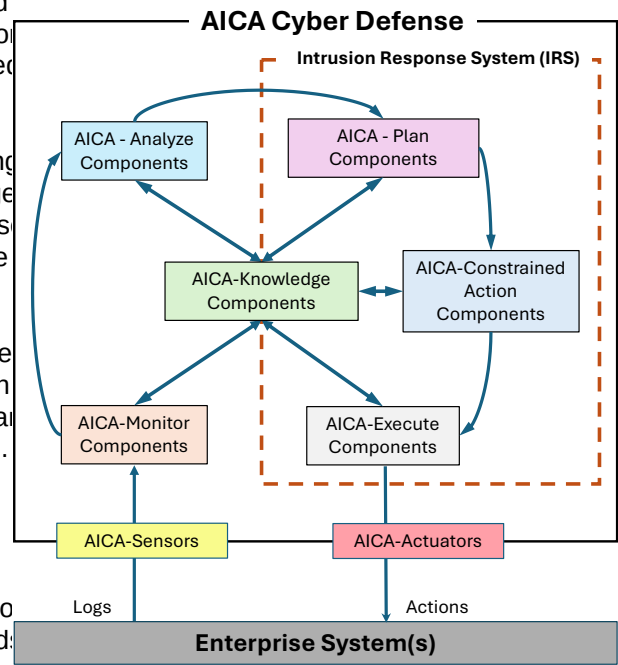


Figure 3: AICA Prototype - Self Adaptive-Autonomic Computing System based MAPE-K [16](SA-ACS) framework implementation. The IRS components are responsible for recovering the enterprise system(s) to its desired state in the event of a security breach. The prototype interacts with enterprise systems via the percepts and actuators. The former gathers the logs, while the latter executes the breached enterprise system(s). The IRS-Plan component uses the logs and the rules to create a computation model. The IRS-Constrained Action component determines the final breach mitigation action(s) following RoE.

III. INTRUSION RESPONSE SYSTEM KNOWLEDGE GRAPH (IRSKG) ONTOLOGY SCHEMA

In this section, we introduce a unified ontology to capture the semantic relationships among different components of an Intrusion Response System (IRS). In addition, we create a schema called IRSKG to represent the ontology in a data structure. Our schema, shown in Figure 4, has been designed to represent disparate enterprise system information such as system logs, system monitoring information, chat conversation logs, intrusion response rules, and response computation model input data. To the best of our knowledge, there is no publicly available ontology that helps represent the above mentioned data for Intrusion Response Systems (IRSs). To build our IRKG ontology we utilize the PG specification (See Section II-A) over *RDF* because of the following primary reasons:

- PG is widely adopted in cyber security domain [17], and is more suitable for dynamic datasets [18].
- PG uses the flexible and extensible JSON format, unlike *RDF*, which uses XML.
- Information retrieval semantic standard is available for PG [19].

In our IRSKG case studies (Section IV), we utilize Neo4J [20], a PG, to demonstrate our ontology implementation. Next, we explain the IRSKG ontology schema in detail. For simplicity we only discuss three enterprise system information schema. The following three subsections describe the schema for the enterprise systems logs, IRS rules, and the computation model inputs. Table I summarizes the used set of notations.

A. Enterprise System Log Schema

In this section, we create a schema that captures the logs of heterogeneous enterprise systems. Typically, the logs are single-line text entries and are better suited for anything but unstructured textual repository format. We choose a graphical representation model to relate information visually. The IRSKG schema uses the graph notations and represents a ‘Graph’ ($\mathcal{G}$) as a set of ‘vertices’ ($\mathcal{V}$) and ‘edges’ ($\mathcal{E}$) as shown in the Equation 1a. We use the PG model technique (see section II-A) and thus each vertex and edges have one or more ‘properties’ along with a ‘label’ that identifies the vertex or edge as shown in the Equations 1b and 1c respectively.

$$\mathcal{G} = (\mathcal{V}, \mathcal{E}) \mid \mathcal{E}_{i,j} \in \mathcal{E}, \mathcal{V}_i \in \mathcal{V} \quad (1a)$$

$$\mathcal{V}_i = \{\mathcal{L}(\mathcal{V}_i), \mathcal{P}(\mathcal{V}_i)\} \quad (1b)$$

$$\mathcal{E}_{i,j} = \{\mathcal{L}(\mathcal{E}_{i,j}), \mathcal{P}(\mathcal{E}_{i,j})\} \quad (1c)$$

We demonstrate an implementation in Section IV-A.

Table I: IRSKG schema notations represent disparate enterprise system information such as system logs, system monitoring information, chat conversation logs, intrusion response rules, and response computation model input data.

Symbol	Description
$\mathcal{G}$	The entire Graph database
$\mathcal{V}$	The set of all vertices in the database.
$\mathcal{E}$	The set of all edges in the database
$\mathcal{V}_i$	The i th vertex.
$\mathcal{E}_{i,j}$	The edge between the $\mathcal{V}_i$ and $\mathcal{V}_j$
$\mathcal{L}(\mathcal{V}_i)$	The label of $\mathcal{V}_i$.
$\mathcal{P}(\mathcal{V}_i)$	The property key-value dictionary of $\mathcal{V}_i$
$\mathcal{L}(\mathcal{E}_{i,j})$	The label of $\mathcal{E}_{i,j}$ edge.
$\mathcal{P}(\mathcal{E}_{i,j})$	The property key-value dictionary of $\mathcal{E}_{i,j}$ edge
$\mathcal{R}$	Set of all Rules of Engagement
$\mathcal{R}_i$	i th Rule of Engagement
$\mathcal{V}_{a b}(\mathcal{R}_i)$	Vertex a or b of rule $\mathcal{R}_i$
$\mathcal{L}(\mathcal{V}_a(\mathcal{R}_i))$	Label of the Vertex a of $\mathcal{R}_i$
$\mathcal{P}(\mathcal{V}_a(\mathcal{R}_i))$	Property of the Vertex a of $\mathcal{R}_i$
$\mathcal{E}(\mathcal{R}_i)$	Edge between $\mathcal{V}_a$ and $\mathcal{V}_b$ of $\mathcal{R}_i$
$\mathcal{L}(\mathcal{E}(\mathcal{R}_i))$	Label of edge between $\mathcal{V}_a$ and $\mathcal{V}_b$ of $\mathcal{R}_i$
$\mathcal{P}(\mathcal{E}(\mathcal{R}_i))$	Property of edge between $\mathcal{V}_a$ and $\mathcal{V}_b$ of $\mathcal{R}_i$
$\mathcal{R}^t \mid \mathcal{R}^i \in \mathcal{R}^{ti}$	A meta-template that different enterprise systems follow and ultimately all $\mathcal{R}_i$ are compliant to.
$\mathcal{R}^{tk}$	A template for a specific enterprise system (e.g. A web enterprise system).

B. Intrusion Response System Rules Schema

We create a schema to represent all the IRS governing rules. The rules are the instructions that tell the IRS how to thwart a security breach. The rules comprise two primary categories: the first category is conditions that trigger the IRS to take action on the breach, and the second is constraints that overwrite the IRS actions deemed unsafe to the system. Usually, these rules follow different schemas and are stored in various file formats (JSON, XML, YARA specification, etc.) IRSKG enables a unified graph schema to represent the rules that show the relationship among the components of the various rules. Next, we describe the IRS governing rules graph notations, their semantics, templates, and constraints, followed by their illustration.

1) Rules Graph Notation: RoEs influence the AICA IRS Plan Components ML models that predict an action or a set of actions to thwart the security breach and restore the enterprise systems to their predefined desired state. We denote RoE as a set of rules $\mathcal{R}$, as shown in the Equation 2a. $\mathcal{R}_i$ has two vertices and an edge connecting those vertices as shown in Equation 2b. We represent the vertices as $\mathcal{V}_{a|b}(\mathcal{R}_i)$, with a label, $\mathcal{L}(\mathcal{V}_a(\mathcal{R}_i))$, and a property set, $\mathcal{P}(\mathcal{V}_a(\mathcal{R}_i))$, as shown in Equation 2c. In addition, we express the edge between $\mathcal{V}_a(\mathcal{R}_i)$ and $\mathcal{V}_b(\mathcal{R}_i)$ as $\mathcal{E}(\mathcal{R}_i)$, with a label $\mathcal{L}(\mathcal{E}(\mathcal{R}_i))$ and a property set $\mathcal{P}(\mathcal{E}(\mathcal{R}_i))$ as shown in Equation 2d. The property set is a key, value pair and can have any arbitrary number of such pairs. However, the organization

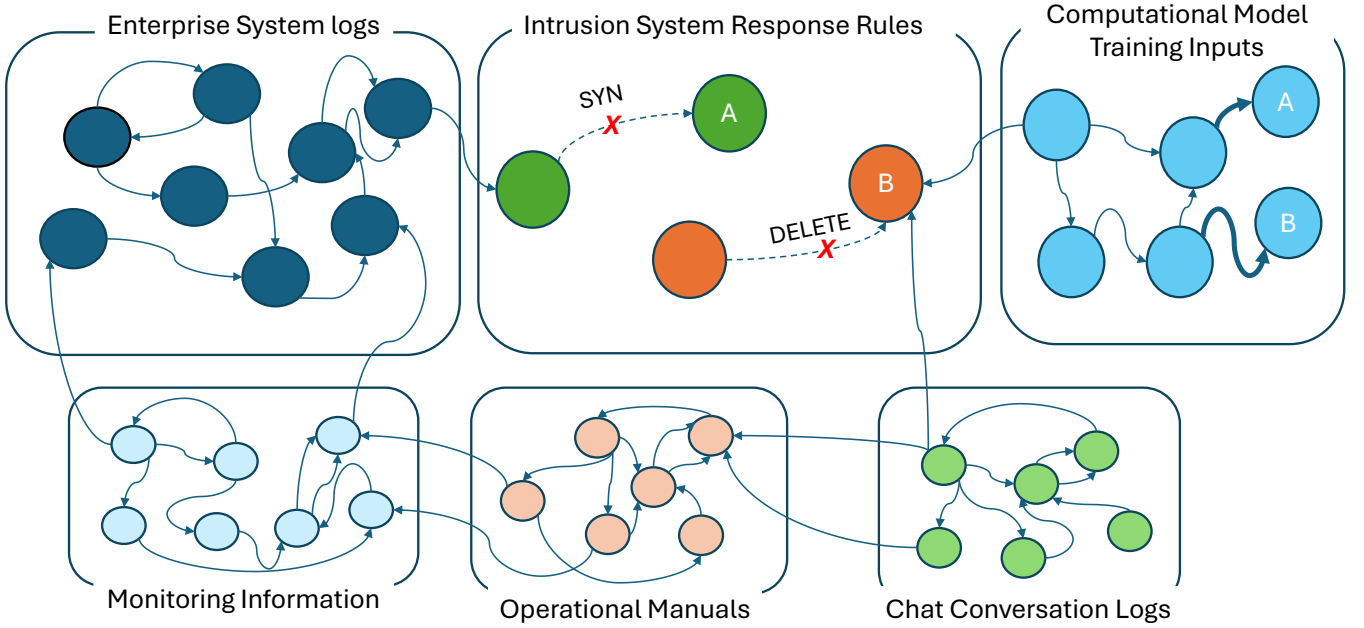


Figure 4: Illustration of IRSKG: A Graph-based model that represents enterprise system information such as System logs, System monitoring logs, Chat conversation logs, IRS rules, and input data for the computational model training.

admin constrains the property set by defining a template (2) that all $\mathcal{R}_i$ must comply with. The logic can be related to the inheritance concept in object-oriented programming to improve understanding.

$$\mathcal{R} = \{\mathcal{R}_i \mid \forall 0 < i < n\} \quad (2a)$$

$$\mathcal{R}_i = \{\mathcal{V}_a(\mathcal{R}_i), \mathcal{E}(\mathcal{R}_i), \mathcal{V}_b(\mathcal{R}_i)\} \quad (2b)$$

$$\mathcal{V}_a(\mathcal{R}_i) = \{\mathcal{L}(\mathcal{V}_a(\mathcal{R}_i)), \mathcal{P}(\mathcal{V}_a(\mathcal{R}_i))\} \quad (2c)$$

$$\mathcal{E}(\mathcal{R}_i) = \{\mathcal{L}(\mathcal{E}(\mathcal{R}_i)), \mathcal{P}(\mathcal{E}(\mathcal{R}_i))\} \quad (2d)$$

2) *Rule Semantic*: The $\mathcal{R}_i$ captures ‘who can do what in which resource’. The ‘who’ and ‘which’ are vertices and capture the source and destination computers. The ‘what’ and ‘which’ are vertices, $\mathcal{V}_a(\mathcal{R}_i)$ and $\mathcal{V}_b(\mathcal{R}_i)$ respectively. We capture the computer $\mathcal{V}$ as the vertex label, namely, $\mathcal{L}(\mathcal{V}_a(\mathcal{R}_i))$ and $\mathcal{L}(\mathcal{V}_b(\mathcal{R}_i))$. However, our schema is flexible; it can contain additional properties such as computer name, computer location, asset name, asset tag, etc. We capture these in $\mathcal{P}(\mathcal{V}_a(\mathcal{R}_i))$ and $\mathcal{P}(\mathcal{V}_b(\mathcal{R}_i))$. The ‘what’ is a verb that the ‘who’ wants to carry on ‘which’ resources. We represent the ‘what’ as an edge $\mathcal{E}(\mathcal{R}_i)$ between $\mathcal{V}_a(\mathcal{R}_i)$ and $\mathcal{V}_b(\mathcal{R}_i)$. The $\mathcal{L}(\mathcal{E}(\mathcal{R}_i))$, the label captures the verb. The $\mathcal{E}(\mathcal{R}_i)$ relationship also captures the ‘constraint’ that $\mathcal{E}(\mathcal{R}_i)$ should follow as laid out by the organization administrators with concrete examples later in the Case Study Section IV.

3) *Rule constraint*: We further define the ‘rule constraints’ that organization administrators define to ‘allow’ or ‘deny’ certain graph relationships. As an illustration, a certain path from vertex $\mathcal{V}_a(\mathcal{R}_i)$ to vertex $\mathcal{V}_b(\mathcal{R}_i)$ with an edge $\mathcal{E}(\mathcal{R}_i)$ is not allowed. We capture this value as a property in $\mathcal{P}(\mathcal{E}(\mathcal{R}_i))$ with a key ‘constraint’. We use this while training the GNN to give a special weight to the relationships of the graph. We aim to have the GNN predict the appropriate action, either deny or allow at model inference time. In addition, our notations are flexible to accommodate additional constraints as needed by the enterprise systems.

4) *Rules Template*: We use ‘meta-template’ and specific ‘templates’ for each enterprise system that each rule should comply with. The ‘meta-template’ governs all enterprise system templates and hence all rules. Each rule must follow an enterprise system template that also adheres to the meta template and introduces further semantics specific to that enterprise system. We define $\mathcal{R}^t$ as the meta template that adheres to the $\mathcal{R}_i$ as we explain in Equation 2b. We define enterprise system templates as $\mathcal{R}^k$. The organization administrator specifies the necessary rule semantics in the enterprise system template. Each enterprise system has precisely one enterprise system template. All instances of an enterprise system use the same system template. Each enterprise system template, $\mathcal{R}^{tk}$, defines a rule set $\mathcal{R}^{tk(j)}$. Two enterprise systems differ at the least by ‘what’ (verb) $\mathcal{E}(\mathcal{R}_i)$ allowed on them. We further explain the template mechanism with concrete examples later in the Case Study Section IV.

5) *Rules illustration*: We illustrate a RoE that uses our IRSKG notation in this section. One rule $\mathcal{R}_1$ could represent a consolidated set of rules $\mathcal{R}_i^{t1}$. As an illustration, an administrator might want to deny remote connectivity from any source IP to a critical network asset such as a physical router in a Network Infrastructure management enterprise system. We represent such a rule with the source IP as the $\mathcal{L}(\mathcal{V}_a(\mathcal{R}_1))$ of the vertex $\mathcal{V}_a(\mathcal{R}_1)$, the destination IP as the $\mathcal{L}(\mathcal{V}_b(\mathcal{R}_1))$ of the vertex $\mathcal{V}_b(\mathcal{R}_1)$, and the connectivity 'SYN' as $\mathcal{L}(\mathcal{E}(\mathcal{R}_1))$ with a property 'constraint' having value 'deny' as shown in the Equation 3. The equation adheres to the model we describe in Section 2a. Moreover, it also adheres to the semantics we define in Section III-B2 where 'who' maps to $\mathcal{L}(\mathcal{V}_a(\mathcal{R}_1))$, 'which' maps to $\mathcal{L}(\mathcal{V}_b(\mathcal{R}_1))$, 'what' maps to the vertex $\mathcal{L}(\mathcal{E}(\mathcal{R}_1))$, and the property 'constraint' with value 'deny' is represented in $\mathcal{P}(\mathcal{E}(\mathcal{R}_1))$ as $\{\text{"constraint"} : \text{"deny"}\}$.

$$\mathcal{R}_1 = \{\mathcal{V}_a(\mathcal{R}_1), \mathcal{E}(\mathcal{R}_1), \mathcal{V}_b(\mathcal{R}_1)\} \quad (3a)$$

$$\begin{aligned} \mathcal{V}_a(\mathcal{R}_1) &= \{\mathcal{L}(\mathcal{V}_a(\mathcal{R}_1)), \mathcal{P}(\mathcal{V}_a(\mathcal{R}_1))\} \\ &| \mathcal{L}(\mathcal{V}_a(\mathcal{R}_1)) \text{ "any"}, \mathcal{P}(\mathcal{V}_a(\mathcal{R}_1)) = \{\dots\} \end{aligned} \quad (3b)$$

$$\begin{aligned} \mathcal{V}_b(\mathcal{R}_1) &= \{\mathcal{L}(\mathcal{V}_b(\mathcal{R}_1)), \mathcal{P}(\mathcal{V}_b(\mathcal{R}_1))\} \\ &| \mathcal{L}(\mathcal{V}_b(\mathcal{R}_1)) = \text{"1.2.3.4"}, \mathcal{P}(\mathcal{V}_b(\mathcal{R}_1)) = \{\dots\} \end{aligned} \quad (3c)$$

$$\begin{aligned} \mathcal{E}(\mathcal{R}_1) &= \{\mathcal{L}(\mathcal{E}(\mathcal{R}_1)), \mathcal{P}(\mathcal{E}(\mathcal{R}_1))\} | \mathcal{L}(\mathcal{E}(\mathcal{R}_1)) = \text{"SYN"}, \\ &\mathcal{P}(\mathcal{E}(\mathcal{R}_1)) = \{\text{"constraint"} : \text{"deny"}\} \end{aligned} \quad (3d)$$

In Section IV-B we demonstrate schema implementation.

C. Response Computation Model Input Data Schema

In addition to the logs and the rules, IRSKG creates a schema for the input data that the IRS uses to prepare a response computational model. IRS uses the model to create an action to thwart security breaches. We use a graph neural network (GNN) as a machine learning model to demonstrate the transformation concept. One can change to any other format suitable for a different machine learning model. Next, we elaborate on IRSKG notations on this ML model input data in this section.

We aggregate the cumulative outbound and inbound connections from and to from each vertex $\mathcal{V}_i$, and represent them in a property in $\mathcal{P}(\mathcal{V}_i)$, 'count' as shown in the Equation 4a. In addition, we also represent the total connections between two vertices, $\mathcal{V}_i$, and $\mathcal{V}_j$ in their edge, $\mathcal{E}_{i,j}$, in a property in the dictionary, $\mathcal{P}(\mathcal{E}_{i,j})$, called 'count' shown in Equation 4b.

$$\mathcal{P}_{count}(\mathcal{V}_i) = \text{deg}(\mathcal{V}_i) | \text{deg} = \text{degree of vertex } \mathcal{V}_i \quad (4a)$$

$$\mathcal{P}_{count}(\mathcal{E}_{i,j}) = \mathcal{P}_{count}(\mathcal{V}_i) + \mathcal{P}_{count}(\mathcal{V}_j) \quad (4b)$$

Next, we provide data transform examples that is needed to create a GNN model, using Equation 4a to calculate $\mathcal{P}_{count}(\mathcal{V}_i)$. Its value is either a cumulative count value as explained by the equation or is a hyper-parameter. For the latter, for example, we transform the constraint rule defined in the Equation 3 as follows: the rule $\mathcal{R}_1$, with a vertex label $\mathcal{L}(\mathcal{V}_1(\mathcal{R}_1))$ of 'any' with an edge $\mathcal{E}(\mathcal{R}_1)$ with a property key and value pair represented in $\mathcal{P}(\mathcal{E}(\mathcal{R}_1))$ as $\{\text{"constraint"} : \text{"deny"}\}$ to a $\mathcal{P}_{count}(\mathcal{V}_1) = -1000000$. The -1000000 value, is a data model hyper-parameter set at the design time, makes the network ignore the edge between any vertex to the vertex $\mathcal{V}_a(\mathcal{R}_1)$ as shown in the Equation 5. The value has to be a sufficiently large negative value. The absolute value depends on the enterprise systems transformed graph $\text{maximum} |\mathcal{P}_{count}(\mathcal{V}_i)|$. We use GNN, so we must adopt this mechanism to define large negative values. However, one can change the mechanism to a different one based on the technique suitable for their IRS of choice.

$$\begin{aligned} \mathcal{P}_{count}(\mathcal{V}_i) &= -1000000 | \mathcal{L}(\mathcal{V}_i(\mathcal{R}_1)) = \text{"any"}, \\ \mathcal{P}(\mathcal{E}(\mathcal{R}_1)) &= \{\text{"constraint"} : \text{"deny"}\} \end{aligned} \quad (5)$$

We demonstrate an implementation in Section IV-C.

IV. CASE STUDY OF THE KNOWLEDGE GRAPH FOR INTRUSION RESPONSE SYSTEM (IRS)

This section demonstrates an IRSKG implementation that translates the abstract schema semantics created in section III to software artifacts. We do so for a cyber defense case study, a Network Infrastructure Management (NIMS) enterprise system. Furthermore, we use the AICA prototype (see section II-B) as the software stack to demonstrate the IRSKG representation. Moreover, we demonstrate the IRS rules, specifically the RoE 'constraints' rules, as explained in Section III-B. In addition, we chose a GNN that the IRS uses as a response computation model (see section III-C). Fig. 5, demonstrates a generalized IRSKG of a typical network system, RoEs that govern the network path between hosts, and the model input to train the GNN. We choose Neo4J [20] to demonstrate a concrete IRSKG schema semantic implementation.

Next, we demonstrate the IRSKG implementation of the NIMS that uses Graylog system logs in three tasks. In each task, we represent the subject in IRSKG, followed by its concrete implementation in Neo4J. We handle Graylog in the first task. We show raw logs, followed by their IRSKG representation and Neo4J implementation. In the second task, we illustrate an IRS rule that denies modifying the router entries. Finally, we show an IRS GNN computation model input that the IRS uses to formulate a response to thwart security breaches. We demonstrate the three above tasks in detail in the following three subsections.

A. Enterprise System Network Logs Graph Schema

This section demonstrates the implementation of the IRSK enterprise system log schema (see section III-A) in Neo4J. We store NIMS logs in Neo4J, which complies with the IRSK enterprise system schema. The current implementation of AICA prototype uses Graylog as a Security Information and Event Management (SIEM) to consolidate logs from sensors in the environment for further IRS use. We use the logs to demonstrate our IRSKG implementation. N_i represents the sources and destinations of a network log entry. E_{ij} represents the action that the source wants to take on the destination. For example, *SYN* is an action when a source sends a connection request to a destination. We represent the activity as the source node, NEP_1 , has $E_{1,2}$ relationship of type *SYN* with a destination NEP_2 .

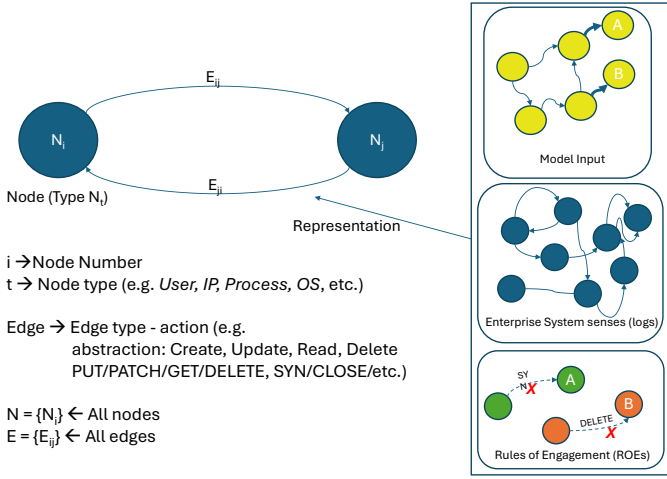


Figure 5: A generalized Network System IRSKG: represents network logs, IRS rules, and the computation model input to train a GNN. For example, for the network logs, the IRSKG Nodes, N_i , represents the source and the destination. Edges, E_{ij} , represents the action the source wants to take on the target, e.g. *SYN* represents an action when a source wants to connect with a destination.

```
1 [2023 10 25 11:10:45] 192.168.1.100 >
  192.168.1.101: TCP SYN
2 [2023 10 25 11:10:46] 192.168.1.101 >
  192.168.1.100: TCP SYNACK
3 [2023 10 25 11:10:47] 192.168.1.100 >
  192.168.1.101: TCP ACK
4 [2023 10 25 11:10:48] 192.168.1.101 >
  192.168.1.100: TCP ACK
```

Listing 1: Sample raw Network Management System GrayLog entries. The entries show the communication between the source, 192.168.0.100 and the destination, 192.168.0.101. The source sends a SYN to the destination; it then sends back SYN-ACK to the source.

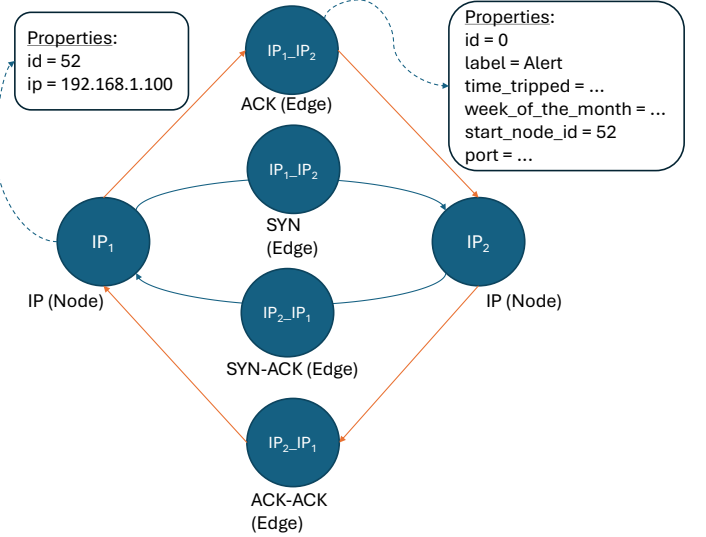


Figure 6: A Network Management Enterprise System IRSKG with sample raw Graylog entries as shown in Listing 2 that complies with IRSKG schema as shown in Fig. 5. The figure shows that source IP_1 , has a property $\mathcal{L}(\mathcal{V}_1) = ip$ with a value $\mathcal{P}(V_i) = \{ip : 192.168.1.100\}$ has relationships, *SYN* and *SYN-ACK*, with destination IP_2 with IP property value 192.168.1.101.

The destination responds to the source with SYN-ACK. Next, the source sends an ACK to the destination, and the latter also reciprocates with an ACK. We represent the log information in IRSKG as shown in Fig. 6 per the schema created in Section III-A.

```
{
  "type": "node", "id": "0", "labels": ["IP1"], "properties": {
    "ip": "192.168.1.100"
  }
}, {
  "type": "node", "id": "1", "labels": ["IP2"], "properties": {
    "ip": "192.168.1.101"
  }
}, {
  "type": "relationship", "id": "0", "label": "SYN", "start": {
    "id": "0", "properties": {
      "time": "23:10:45",
      "time_month": "10", "time_year": "2023", "time_date": "25",
      "end": {
        "id": "1", "properties": {

```

Listing 2: IRSKG Graylog Network Management system implementation of the raw entries as shown in Listing 1 and Figure 6. The graph entries show two nodes with $id=0$ and $id=1$ with their corresponding ips representing the source and the destination, 192.168.1.100 and 192.168.1.101. There is a relationship between the two of type *SYN* and with properties that store the time when source sent a SYN request to the destination.

B. Enterprise System Network Rules Graph Schema

We show in the Listing 1 a few raw Graylog entries from the Router. The Graylog listing shows that source 192.168.1.100 sends a connect SYN, to the destination denies any machine (source) to modify the router with an IP 10.10.10.10. SYN is the rst byte sent by the source. We represent the

Figure 7 and store the rule in the Neo4J IRSKG implementation as shown in Listing 3. The node type of the source and destination nodes, represented as $\mathcal{V}_a(\mathcal{R}_1) = NEP_1$ and $\mathcal{V}_b(\mathcal{R}_1) = NEP_2$, are `NetworkEndpoint` type. They have property name `ip_address` values as `*` and `10.10.10.10` represented in $\mathcal{P}(\mathcal{V}_a(\mathcal{R}_1))$ and $\mathcal{P}(\mathcal{V}_b(\mathcal{R}_1))$ respectively. Furthermore, NEP_1 and NEP_2 has `id` property, specific to Neo4J software, as `27550` and `27551` respectively. The edge $\mathcal{E}_{1,2}$ of type `relationship`, between NEP_1 (start) and NEP_2 (end) has a label $\mathcal{L}(\mathcal{E}(\mathcal{R}_1)) = COMMUNICATES_TO$ with properties such as $\mathcal{P}_{constraint}(\mathcal{E}(\mathcal{R}_1))$ as `deny`.

```

1 {
2   "type": "node",
3   "id": "27551",
4   "labels": [
5     "NEP1"
6   ],
7   "properties": {
8     "ip_address": "*"
9   }
10 }
11 {
12   "type": "node",
13   "id": "27550",
14   "labels": [
15     "NEP2"
16   ],
17   "properties": {
18     "ip_address": "10.10.10.10"
19   }
20 }
21 {
22   "type": "relationship",
23   "id": "0",
24   "label": "COMM1",
25   "start": {
26     "id": "27551",
27     "labels": [
28       "NEP1"
29     ],
30     "properties": {
31       "ip_address": "*"
32     }
33   },
34   "end": {
35     "id": "27550",
36     "labels": [
37       "NEP2"
38     ],
39     "properties": {
40       "ip_address": "10.10.10.10"
41     }
42   },
43   "properties": {
44     "action": "ADD",
45     "constraint": "deny",
46     "id": "6ec4f95c-f4e3-4516-92c1-172cec275696"
47   }
48 }
49 }

```

Listing 3: Neo4J IRSKG IRS rule that prevents any machine from modifying the router with `constraint deny` for the `action ADD` as shown in Figure 7. The source is represented as a node with `id=27751` having `ip_address = *`. The router is represented as a node with `id=27750` having `ip_address = 10.10.10.10`. The relationship is represented as an edge with an `id=0` with `label=COMMUNICATES_TO`.

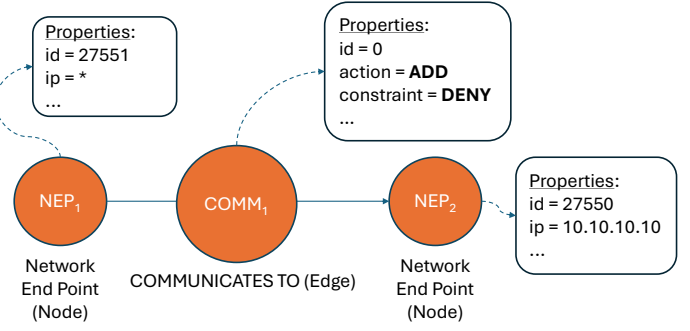


Figure 7: IRSKG for Network Management System constraint rule that denies any source, represented as `*` to modify the router represented as `10.10.10.10` with a rule to `constraint` to deny (with a value `DENY`) for an `action = ADD` as shown in Listing 3 and that complies with IRSKG ontology schema as shown in Figure 5.

C. Enterprise System Network Computational Model Input Graph Schema

Finally, we demonstrate the IRSKG response computational model input (see Section III-C) implementation in this section. The IRS uses this data in the NIMS case study to train a GNN model. Next, we explain the GNN computational model input schema using the Graylogs, as shown in Fig. 6 and constraint rules, as shown in Fig. 7. We illustrate how the input Gray logs, as shown in the Listing 1, are transformed to the GNN input data schema as shown in the Listing 4. We calculate the `count` property value, $\mathcal{P}_{count}(\mathcal{V}_i)$ as 4, following the Equation 4a because there are two nodes, where the `'192.168.1.100'` appears as either the `source` or `destination` address in Listing 1. The source node is represented as IP_1 and the destination node as IP_2 as shown in Figure 6. Similarly, we assign the same `count` value to the `ip '192.168.1.101'` as shown in the Listing 4. Moreover, we calculate the property value, $\mathcal{P}_{count}(\mathcal{E}_{i,j})$ as two of the edges $\mathcal{E}_{1,2}$ between IP_1 and IP_2 , as shown in the Listing 5 abiding to the Equation 4b.

```

1 {
2   "type": "node",
3   "id": "2891",
4   "labels": [
5     "IP1"
6   ],
7   "properties": {
8     "ip_address": "192.168.1.100",
9     "count": 2
10  }
11 }
12 {
13   "type": "node",
14   "id": "2892",
15   "labels": [
16     "IP2"
17   ],
18   "properties": {
19     "ip_address": "192.168.1.101",
20     "count": 2
21   }
22 }

```


Listing 4: Neo4J IRSKG computational model input implementation for vertexes IP_1 with $ip_address = 192.168.1.100$ and IP_2 with $ip_address = 192.168.1.101$: since these IPs appear twice in Graylog Listing 1. Hence $\mathcal{P}_{count}(V_i)$ as 4 as represented in properties count based on Equation 4a.

```

1 {
2   "type": "relationship",
3   "id": "878",
4   "label": "COMMUNICATES_TO",
5   "start": {
6     "id": "2891",
7     "labels": [
8       "IP1"
9     ],
10    "properties": {
11      "ip_address": "192.168.1.100"
12    }
13  },
14  "start": {
15    "id": "2892",
16    "labels": [
17      "IP2"
18    ],
19    "properties": {
20      "ip_address": "192.168.1.101"
21    }
22  },
23  "properties": {
24    "count": 2,
25    "id": "6ec4f95c-f4e3-4516-92c2-172cec275696"
26  }
27 }

```

Listing 5: Neo4J IRSKG computational model input implementation for the Edge $E_{1,2}$ with two nodes $-IP_1$ with $ip_address = 192.168.1.100$ and IP_2 with $ip_address = 192.168.1.101$: since these IPs appear twice in Graylog Listing 1. Hence $\mathcal{P}_{count}(E_{1,2})$ as 4 as represented in properties count based on Equation 4b.

```

1 {
2   "type": "node",
3   "id": "27550",
4   "labels": [
5     "NEP2"
6   ],
7   "properties": {
8     "ip_address": "10.10.10.10",
9     "count": 1000000
10  }
11 }

```

Listing 6: IRSKG GNN computational model input data schema representing transformed vertex, the router NEP_2 with $ip_address=10.10.10.10$ based on (shown in Listing 3 and in Figure 7) has a 'count' property value of -1000000 per Equation 5.

Finally, we transform the IRSKG Gray logs using the constraint IRSKG rules to the IRSKG GNN computational input IRSKG. As explained in Section III-B, an enterprise administrator creates a constraint rule R_1 , to prevent any

source IP from connecting to a critical infrastructure piece, the router, by adhering to the rules of engagement semantics as illustrated in the Equation 5 and demonstrated in Listing 3 and in Figure 7 R_1 prevents any machines from connecting to the router, thus denying them the ability to add new rules to the router. The constraint rule transforms the router vertex and the edge to vertex from any source IP and assigns the edge property to -1000000. The transformed GNN computational model input IRSKG schema is shown in Fig. 8. The IRSKG Neo4J implementations for the Vertex and the Edge are shown in the Listing 6 and 7.

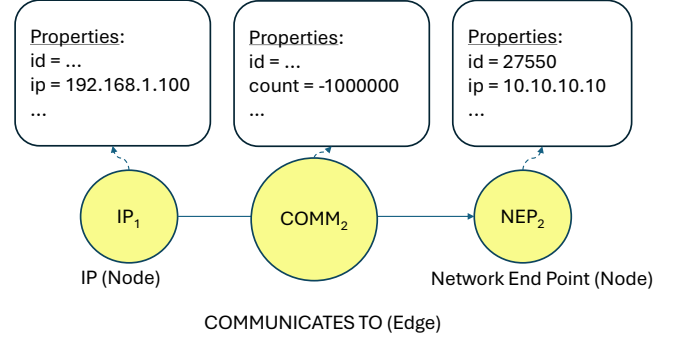


Figure 8: IRSKG GNN computational model input data schema using the Gray log and the constraint rule, that denies modifying the router. Graylog IRSKG is shown in the Fig. 6 and its Neo4J implement is shown in the Listing 1. The constraint IRSKG is shown in Fig. 7 and its Neo4J implementation is shown in the Listing 3. Based on R_1 , we represent $\mathcal{P}_{count}(E_{1,2})$ as -1000000 to the Edge $E_{1,2} = COMM_2$, between P_1 with $ip_address = 192.168.1.100$ and NEP_2 with $ip_address = 10.10.10.10$.

```

1 {
2   "type": "relationship",
3   "id": "878",
4   "label": "COMM2",
5   "start": {
6     "id": "2891",
7     "labels": [
8       "IP1"
9     ],
10    "properties": {
11      "ip_address": "192.168.1.100"
12    }
13  },
14  "start": {
15    "id": "27550",
16    "labels": [
17      "NEP2"
18    ],
19    "properties": {
20      "ip_address": "10.10.10.10"
21    }
22  },
23  "properties": {
24    "count": 1000000,
25    "id": "6ec4f95c-f4e3-4516-92c2-172cec275696"
26  }
27 }

```

Listing 7: IRSKG GNN computational model input data schema representing transformed edge (as shown in Figure 8) MM_2 , between the router, NEP_2 with ip-address=10.10.10.10 and vertex, IP_1 with ip-address=192.168.1.100 based on (shown in Listing 3 and in Figure 7) has count property value of -1000000 per Equation 5.

V. CONCLUSION

The goal of the paper is to introduce a novel schema, called IRSKG, for capturing information to enhance cyber defense Intrusion Response Systems (IRSs). The schema accomplishes this by enabling: faster onboarding of new enterprise systems, brisker IRS rules management, and faster input data transformations to continuously train computation models to thwart security breaches. Additionally, IRSKG is designed to be adaptable to the evolving cyber threat landscape and allows the onboarding of new configurable structures. This schema represents enterprise system information, including Enterprise system logs, IRS rules, computation model input data, and chat conversation history. We chose a Network Infrastructure Management system as a case study using the AICA Prototype software for the demonstration. Using IRSKG, we represented Graylog network logs, IRS rules that govern the network path between hosts, and the model input to train the GNN. We considered GNN for demonstration due to the graphical nature of the data structure; however, one could use any technique and AI/ML model type to implement IRSKG. Moreover, one could choose a different prototype and a case study to prototype IRSKG. This unified and robust approach allows streamlined automated intrusion response with collaborative information sharing and explainability. In the future, we plan to automate our approach further by incorporating a set of programming APIs and tools to provide additional methods to interact with the IRSKG schema-compliant data. We want to use the APIs to ingest enterprise systems' logs, IRS rules, and computation model input data to use the tools and manage the data by visualizing the schema.

VI. ACKNOWLEDGEMENTS

The work was supported by PATENT Lab at the Department of Computer Science and Engineering, Mississippi State University. The views and conclusions are those of the authors.

REFERENCES

- [1] A. Kott, "Autonomous intelligent cyber defense agent (aica)."
- [2] D. J. Ragsdale, C. Carver, J. W. Humphries, and U. W. Pooch, "Adaptation techniques for intrusion detection and intrusion response systems," in *Smc 2000 conference proceedings. 2000 IEEE international conference on systems, man and cybernetics: cybernetics evolving to systems, humans, organizations, and their complex interactions* (cat. no. 0), vol. 4. IEEE, 2000, pp. 2344–2349.
- [3] V. Cardellini, E. Casalicchio, S. Iannucci, M. Lucantonio, S. Mittal, D. Panigrahi, and A. Silvi, "irs-partition: An intrusion response system utilizing deep q-networks and system partitions," *SoftwareX*, vol. 19, p. 101120, 2022.
- [4] S. Mitra, T. Chakraborty, S. Neupane, A. Piplai, and S. Mittal, "Use of graph neural networks in aiding defensive cyber operations," *arXiv preprint arXiv:2401.05680*, 2024.
- [5] Y. Tian, "The world of graph databases from an industry perspective," *ACM SIGMOD Record*, vol. 51, no. 4, pp. 60–67, 2023.
- [6] R. Syed, "Cybersecurity vulnerability management: A conceptual ontology and cyber intelligence alert system," *Information & Management*, vol. 57, no. 6, p. 103334, 2020.
- [7] T. Qamar and N. Z. Bawany, "A cyber security ontology for smart city," *International Journal on Information Technologies & Security*, vol. 12, no. 3, pp. 63–74, 2020.
- [8] O. T. Arogundade, A. Abayomi-Alli, and S. Misra, "An ontology-based security risk management model for information systems," *Arabian Journal for Science and Engineering*, vol. 45, no. 8, pp. 6183–6198, 2020.
- [9] S. Mitra, A. Piplai, S. Mittal, and A. Joshi, "Combating fake cyber threat intelligence using provenance in cybersecurity knowledge graphs," in *2021 IEEE International Conference on Big Data (Big Data)*. IEEE, 2021, pp. 3316–3323.
- [10] T. Developer, "Graph Database Fundamentals," <https://terminusdb.com/blog/graph-database-fundamentals/>, n.d., accessed: 2023-07-23.
- [11] O. Developer, "Graph Developer's Guide for Property Graph," <https://docs.oracle.com/en/database/oracle/property-graph/22.2/spgdl/what-are-property-graphs.html>, n.d., accessed: 2023-12-25.
- [12] R. Angles, "The property graph database model," *AIW*, 2018.
- [13] D. R. W. Group, "Resource Description Framework (RDF)," <https://www.w3.org/RDF/>, 2014, accessed: 2024-09-11.
- [14] N. Shadbolt, T. Berners-Lee, and W. Hall, "The semantic web revisited," *IEEE intelligent systems*, vol. 21, no. 3, pp. 96–101, 2006.
- [15] A. Kott, *Autonomous Intelligent Cyber Defense Agent (AICA): A Comprehensive Guide*. Springer Nature, 2023, vol. 87.
- [16] J. Kephart and D. Chess, "The vision of autonomic computing," *Computer*, vol. 36, no. 1, pp. 41–50, 2003.
- [17] R. Angles, A. Bonifati, S. Dumbrava, G. Fletcher, A. Green, J. Hidders, B. Li, L. Libkin, V. Marsault, W. Martens, et al., "Pg-schema: Schemas for property graphs," *Proceedings of the ACM on Management of Data*, vol. 1, no. 2, pp. 1–25, 2023.
- [18] V. Vetrivel, "Knowledge Graphs: RDF or Property Graphs, Which One Should You Pick?" <https://www.wisecube.ai/blog/knowledge-graphs-rdf-or-property-graphs-which-one-should-you-pick/>, 2022, accessed: 2024-09-11.
- [19] A. Deutsch, N. Francis, A. Green, K. Hare, B. Li, L. Libkin, T. Linddaaker, V. Marsault, W. Martens, J. Michels, et al., "Graph pattern matching in gq and sql/pgq," in *Proceedings of the 2022 International Conference on Management of Data*, 2022, pp. 2246–2258.
- [20] Neo4j, "Neo4j: Graphs for Everyone," <https://github.com/neo4j/neo4j>, 2007, accessed: 2024-09-11.